

VECTOR DATABASES

From Scratch to Production

A Complete Engineer's Guide

Covering Embeddings · HNSW · ANN Search · Production Systems

2025 Edition

Table of Contents

Chapter 1: The Problem — Why Traditional Databases Break

Modern applications need to understand meaning, not just match words. When a user types 'how do I reset my password,' they expect to find articles about 'account recovery' and 'forgot login' — even if those exact words never appear in their query. This is called semantic search, and it fundamentally breaks every assumption that traditional databases were built on.

In this chapter, we walk through exactly why the tools that made databases fast — indexes, B-trees, WHERE clauses — are useless for semantic search. This is not a limitation we can engineer around with clever SQL. It is a mathematical incompatibility that requires a completely different kind of system.

1.1 How Traditional Databases Work

Relational databases achieve their speed through a mechanism called indexing. An index is a sorted auxiliary data structure — usually a B-tree — that maps values to row locations. When you query `WHERE user_id = 12345`, the database does not scan every row. It navigates the B-tree in $O(\log n)$ time and jumps directly to the matching rows.

This is extraordinarily efficient. A table with 100 million rows can be queried in under a millisecond because the index allows the database to skip the vast majority of data.

How a B-tree Index Works

Think of a phone book. To find 'Smith, John' you don't read every entry — you jump to 'S', then 'Sm', then 'Smith'. A B-tree works the same way: each node points to a range of values, letting you discard half the remaining candidates at every step. With 1 billion records, you find any row in about 30 comparisons.

Range queries work similarly. `WHERE created_at > '2025-01-01'` uses a date index to find the first matching row and scans forward linearly from there. The fundamental requirement for all of this: there must be an exact or orderable value to compare against.

1.2 What Is an Embedding?

To understand semantic search we first need to understand embeddings. An embedding is a mathematical representation of meaning. You feed text into an embedding model and it returns a vector — a list of floating-point numbers — where similar meanings produce numerically similar vectors.

- "reset my password" → [0.23, -0.81, 0.44, 0.17, ..., 0.62] (1,536 numbers)
- "forgot login credentials" → [0.21, -0.79, 0.48, 0.14, ..., 0.59] (very similar)
- "quarterly earnings report" → [-0.44, 0.33, -0.71, 0.88, ..., -0.12] (very different)

The key property: distance between vectors corresponds to semantic distance between meanings. Text that means similar things ends up geometrically close in this high-dimensional space, regardless of shared words.

1.3 Why Similarity Is Not Equality

With traditional search you look for equality: WHERE name = 'John'. With embedding-based search you look for proximity: find the 10 vectors most similar to this query vector. Proximity is a fundamentally different operation. There is no index you can build that lets you jump directly to nearest neighbors in high-dimensional space, for a deep mathematical reason.

The Curse of Dimensionality

In low dimensions (2D, 3D), 'nearby' points are actually close.
In high dimensions (1,536D), almost all pairs of points become approximately equidistant from each other. Standard partitioning strategies like KD-trees, which work well up to ~20 dimensions, deteriorate toward full scans as dimensionality grows.

1.4 The $O(n)$ Problem

Without specialized data structures, the only way to find similar vectors is brute-force: compare the query against every stored vector, compute a distance score, sort, return top k. Time complexity is $O(n)$ — linear in dataset size.

Dataset Size	Time per Query (0.0001s each)
10,000 vectors	~1 second
1,000,000 vectors	~100 seconds
10,000,000 vectors	~1,000 seconds (16+ minutes)
100,000,000 vectors	~2.7 hours

And this is for a single query. At 100 concurrent queries per second the numbers become completely impossible. The problem is not hardware — it is the algorithm.

1.5 Why More Indexes Don't Help

A B-tree works because the key space is one-dimensional and totally ordered. Any value is less than, equal to, or greater than any other value. This lets you build hierarchical partitions that eliminate large swaths of data with each comparison.

In 1,536 dimensions there is no equivalent total ordering. A vector is not 'less than' or 'greater than' another vector in any globally meaningful sense. The only meaningful relationship is distance, and computing distance requires looking at all 1,536 dimensions simultaneously.

This is why vector databases exist: not to replace relational databases, but to solve a problem that relational databases were never architected to handle.

Chapter 2: Embeddings and Vector Mathematics

Before we can build or use a vector database, we need a solid understanding of the raw material it stores: embedding vectors. This chapter covers how embeddings are created, what they represent geometrically, and how similarity is measured.

2.1 What Is an Embedding Model?

An embedding model is a neural network trained to map inputs — text, images, audio — into a fixed-length list of floats. The training process ensures that semantically related inputs produce vectors that are close together in the resulting space.

For language, training typically involves predicting missing words (BERT-style), predicting the next token (GPT-style), or contrastive loss that explicitly pushes similar sentences together and dissimilar ones apart. The result encodes rich relationships:

- $\text{king} - \text{man} + \text{woman} \approx \text{queen}$ (the famous word2vec demonstration)
- $\text{Paris} - \text{France} + \text{Germany} \approx \text{Berlin}$
- "I love dogs" and "I adore puppies" land geometrically close
- "I love dogs" and "quarterly tax filings" land far apart

2.2 Popular Embedding Models (2025)

Model	Dimensions	Notes
OpenAI text-embedding-3-small	1,536	Good quality/cost balance
OpenAI text-embedding-3-large	3,072	Highest OpenAI accuracy
Cohere embed-v3	1,024	Strong multilingual support
BGE-M3 (open source)	1,024	Excellent multilingual, free
all-MiniLM-L6-v2	384	Fast, small, runs locally
Google text-embedding-004	768	Matryoshka, truncatable

Critical rule: always use the same model for both indexing and querying. Each model defines its own vector space. Mixing models produces nonsensical results, as if looking up French words in an English dictionary.

2.3 Vector Space Geometry

Think of each embedding as a point in a very high-dimensional space. In 2D, two close points are nearby on a flat map. In 1,536D the same idea holds — but we cannot visualize it. The geometry has two key properties:

- Direction encodes meaning: two vectors pointing in the same direction represent semantically similar content, regardless of magnitude.
- Distance encodes dissimilarity: vectors far apart represent content with different meanings.

2.4 Distance and Similarity Metrics

Cosine Similarity

Measures the angle between two vectors. Ranges from -1 (opposite) to 1 (identical direction). Length-invariant, making it ideal for text.

$$\begin{aligned} \text{cosine_similarity}(A, B) &= (A \cdot B) / (|A| \times |B|) \\ &= \sum(A_i \times B_i) / (\sqrt{\sum(A_i^2)} \times \sqrt{\sum(B_i^2)}) \end{aligned}$$

Dot Product

Sum of element-wise products. Not normalized, so larger-magnitude vectors score higher regardless of angle. When vectors are unit-normalized (many modern models do this), dot product and cosine similarity produce identical rankings — and dot product is faster since no normalization is needed at query time.

$$\text{dot_product}(A, B) = A \cdot B = \sum(A_i \times B_i)$$

Euclidean Distance (L2)

Straight-line distance between two points. Lower = more similar. Sensitive to vector magnitude, making it less suitable for unnormalized text embeddings but natural for image and audio features.

$$L2(A, B) = \sqrt{\sum(A_i - B_i)^2}$$

Metric Selection Guide

- Text embeddings (most use cases): Cosine similarity or Dot Product
- Unit-normalized embeddings: Dot Product (faster, same ranking)
- Image / audio features: Euclidean (L2)
- When in doubt: check your model's documentation for its recommendation

2.5 Chunking: Preparing Text for Embedding

Embedding models accept limited input (512–8,192 tokens). Long documents must be split into chunks. Chunking strategy has a large impact on retrieval quality.

Fixed-Size Chunking

Split every N tokens with M token overlap. Simple and predictable; can split sentences mid-thought.

```
chunk_size = 512 tokens, overlap = 50 tokens
```

Sentence-Level Chunking

Split at sentence boundaries. Preserves semantic units at the cost of variable chunk sizes.

Semantic Chunking

Embed each sentence, group adjacent semantically-similar sentences together. Best retrieval quality, highest compute cost during ingestion.

Hierarchical Chunking

Store embeddings at multiple granularities (sentence, paragraph, section, document). The query layer retrieves at the appropriate level.

2.6 Storage Cost of Embeddings

Each dimension is a 32-bit float (4 bytes):

```
1,536 × 4 bytes = 6,144 bytes ≈ 6 KB per vector
```

- 1 million vectors = ~6 GB raw vector data
- 10 million vectors = ~60 GB
- 100 million vectors = ~600 GB

This is before indexing structures, metadata, or overhead. Storage cost is a first-class concern in vector database design, which is why compression techniques (Chapter 3) are critical at scale.

Chapter 3: Approximate Nearest Neighbor Search

Exact nearest neighbor (NN) search doesn't scale. This chapter explores the solution — approximate nearest neighbor (ANN) search. We examine the core tradeoffs, survey the main algorithmic families, and build intuition for why approximation is not a weakness but a well-understood and controlled design choice.

3.1 Exact vs. Approximate: The Core Tradeoff

Exact NN search guarantees returning the true closest vector. Guaranteeing exactness requires, in the worst case, examining every vector — $O(n)$ time. ANN search relaxes this guarantee. It promises to find the true nearest neighbors most of the time (quantified as 'recall') while doing so dramatically faster.

Key ANN Metrics

Recall@k:	Fraction of true top-k neighbors returned by ANN. Recall@10 = 0.95 means 9.5 of 10 true neighbors found.
QPS:	Queries per second the system can sustain.
p99 Latency:	99th percentile query time (worst case for 99% of queries).
Index build time:	Time to construct the index from scratch.
Index size:	Memory / disk footprint of the index structure.

In practice, ANN with 95%+ recall is indistinguishable from exact search for most applications. Users cannot tell whether result #8 is the 8th or 9th true nearest neighbor. The speed gain, however, can be 100x to 10,000x.

3.2 The Three Algorithmic Families

1. Tree-Based Methods (KD-Tree, Annoy)

Partition the vector space recursively using hyperplane splits. Navigate the tree at query time to find the region containing the query.

- Pros: Simple, interpretable, no training, good up to ~50 dimensions
- Cons: Degrades to full scan above ~50 dimensions due to the curse of dimensionality
- Used by: Annoy (Spotify), early Faiss variants

2. Hash-Based Methods (LSH — Locality Sensitive Hashing)

Random hash functions designed so similar vectors hash to the same bucket. Query by hashing and checking vectors in the same bucket(s).

- Pros: Strong theoretical guarantees; works at large scale

- Cons: Requires many hash tables for high recall; high memory overhead; harder to tune than graph methods in practice
- Used by: Early Pinterest visual search, some fraud detection systems

3. Graph-Based Methods (HNSW, NSG, DiskANN)

Build a navigable graph where each vector is a node connected to its approximate nearest neighbors. Queries navigate the graph greedily toward the query vector.

- Pros: State-of-the-art recall/speed tradeoff; highly tunable; supports fast incremental inserts
- Cons: High memory usage for graph structure; slow builds for large datasets
- Used by: Pinecone, Weaviate, Qdrant, Milvus, pgvector — essentially every modern vector database

Industry Consensus

HNSW has become the dominant ANN algorithm in production vector databases. The ann-benchmarks.com project consistently shows graph-based methods leading the recall/QPS frontier for datasets up to ~10M vectors. For billion-scale datasets, disk-based variants like DiskANN are emerging.

3.3 The Recall-Speed Frontier

Every ANN algorithm exposes tuning parameters that let you slide along a tradeoff between recall and speed. The best algorithms produce a frontier that dominates others — at every recall level, they achieve higher QPS than competitors.

- Sub-10ms latency requirement → tune for speed, accept lower recall (90%+)
- Revenue-critical recommendations → tune for high recall (98%+), accept higher latency
- You cannot be at the top of both axes — any system claiming otherwise is incorrect

3.4 Quantization: Compressing Before Indexing

Quantization is lossy compression applied to vectors before (or during) indexing. It reduces storage, memory, and computation costs at the cost of small accuracy losses.

Scalar Quantization (SQ8)

Each 32-bit float dimension is converted to an 8-bit integer. 4x memory reduction; can be 4x faster due to SIMD int8 instructions. Recall loss is typically under 1%.

|float32 value 0.7341 → uint8 value 188 (mapped linearly to [0, 255])

Product Quantization (PQ)

The vector is split into M subvectors, each quantized separately using a codebook learned from the data. Can achieve 32x+ compression with moderate recall loss.

```
1,536-dim vector → 48 subvectors of 32 dims each  
Each subvector mapped to 1 of 256 codebook entries → 48 bytes total (vs  
6,144)
```

Binary Quantization

Each dimension reduced to a single bit (positive → 1, negative → 0). 64x compression, significant recall loss. Often combined with re-ranking using full vectors to recover accuracy.

3.5 Inverted File Index (IVF)

IVF is a complementary technique often combined with PQ (IVF+PQ). The dataset is first clustered into N groups (using k-means). At query time, the top C closest cluster centroids are identified and search is restricted to vectors in those clusters.

This dramatically reduces the number of distance comparisons at the cost of: (a) an offline clustering training phase, and (b) potential errors when a true nearest neighbor lands in an unvisited cluster. IVF is more commonly used in Faiss for batch analytics than in real-time serving, where HNSW dominates.

Chapter 4: HNSW — The Algorithm in Depth

Hierarchical Navigable Small World (HNSW) is the algorithm at the heart of nearly every modern vector database. It was introduced by Malkov and Yashunin in a 2018 IEEE paper. Understanding it deeply — including the search, insert, and delete operations — is essential for anyone operating vector databases in production.

4.1 The Intuition: Small World Networks

HNSW is inspired by 'small world' networks: graphs where most nodes are not directly connected, but any two nodes can be reached from each other through a small number of hops. The classic example is 'six degrees of separation' — any two people on Earth are linked by at most six social connections.

A small world network has two key structural properties:

- Local clustering: your friends tend to know each other (short-range, dense edges)
- Long-range bridges: you know someone in another country (long-range, sparse edges)

HNSW applies this to vector spaces: it builds a graph where short edges provide local precision and long edges enable global navigation. A greedy search can start anywhere and efficiently navigate to any query vector by following edges that move closer to the target.

4.2 The Hierarchical Structure

The 'H' in HNSW stands for Hierarchical. The graph has multiple layers — think of a road network:

- Layer 0 (ground level): Every vector exists here, connected to its nearest neighbors. Final precision comes from here.
- Layer 1 (local roads): A subset of vectors, connections spanning greater distances.
- Layer 2 (highways): A smaller subset, connections spanning even farther.
- Layer L (top): Typically 1-3 vectors; serves as the search entry point.

A vector's maximum layer is assigned probabilistically at insertion time using an exponential distribution:

$$P(\text{layer} \geq l) = \exp(-l / mL) \quad \text{where } mL = 1 / \ln(M)$$

With default settings ($M=16$, $mL \approx 0.36$):

- 100% of vectors appear in layer 0
- ~13% appear in layer 1
- ~2% appear in layer 2
- ~0.2% appear in layer 3

This geometric thinning mirrors how transportation networks work: many local streets, fewer arterials, fewer highways, a handful of interstate routes.

4.3 HNSW Search Algorithm

Given a query vector q and desired result count k , search navigates the graph top-down through two distinct phases.

Phase 1: Greedy Descent (Layers L to 1)

At each layer above 0, find the node closest to q among the current node's neighbors, move there, and repeat until no neighbor is closer. This is pure greedy hill-climbing. Only one candidate is tracked per layer.

```
function GREEDY_SEARCH(q, entry_point, layer):
    current = entry_point
    improved = true
    while improved:
        improved = false
        for neighbor in neighbors(current, layer):
            if dist(q, neighbor) < dist(q, current):
                current = neighbor
                improved = true
    return current                // closest found at this layer
```

Phase 2: Beam Search (Layer 0)

At layer 0, switch from greedy (single best) to beam search (maintain ef candidates). This wider exploration is what provides recall above the greedy baseline.

```
function BEAM_SEARCH(q, entry_point, ef, layer):
    visited = {entry_point}
    C = MinHeap()                // candidates ordered by dist to q (closest
    first)
    W = MaxHeap(maxsize=ef)      // result set, ordered worst-first
    C.push(entry_point, dist(q, entry_point))
    W.push(entry_point, dist(q, entry_point))

    while C not empty:
        c = C.pop()              // closest unvisited candidate
        f = W.top()              // farthest element in result set
        if dist(q, c) > dist(q, f):
            break                // no candidate can improve result set
        for e in neighbors(c, layer):
            if e not in visited:
                visited.add(e)
                if dist(q, e) < dist(q, f) or len(W) < ef:
                    C.push(e, dist(q, e))
                    W.push(e, dist(q, e))
                    if len(W) > ef: W.pop()    // maintain beam width
    return W
```

Full Search

```
function SEARCH(q, k, ef_search):
    ep = entry_point                // global entry node
    L = max_layer(ep)
    for lc = L downto 1:            // Phase 1
        ep = GREEDY_SEARCH(q, ep, lc)
    candidates = BEAM_SEARCH(q, ep, ef_search, 0) // Phase 2
    return top_k(candidates, k)
```

The ef_search Parameter

ef_search (also called 'ef' or 'search_ef') controls beam width in Phase 2.

ef = 10: Fast, may miss some true nearest neighbors

ef = 100: Slower, near-exact results

ef must always be $\geq k$ (you explore at least as many as you return)

Tuning ef at query time changes recall/speed WITHOUT rebuilding the index.

4.4 HNSW Insert Algorithm

Inserting a new vector is more complex than search — you must not only find the new vector's neighbors but also update those neighbors' lists to include the new vector.

Step-by-Step Insert

1. Assign a random maximum layer l using the exponential distribution.
2. Greedy descent from the global top layer to layer $l+1$: find the closest node at each layer (same as Phase 1 of search). This locates the insertion neighborhood.
3. For each layer from $\min(l, L)$ down to 0: run BEAM_SEARCH with ef_construction candidates; select M neighbors using the diversity heuristic (see below); add bidirectional edges.
4. Prune neighbor lists that now exceed $M_{\max}$ by removing the farthest neighbor(s).
5. If $l >$ current max layer: update the global entry point to this new node.

Insert Pseudocode

```
function INSERT(q):
    l = random_layer()
    L = max_layer(entry_point)
    ep = entry_point

    for lc = L downto l+1:
        ep = GREEDY_SEARCH(q, ep, lc)    // find insertion neighborhood

    for lc = min(l, L) downto 0:
```

```

cands    = BEAM_SEARCH(q, ep, ef_construction, lc)
selected = SELECT_NEIGHBORS(q, cands, M, lc)
for e in selected:
    add_edge(q, e, lc)           // bidirectional
    add_edge(e, q, lc)
    if len(neighbors(e, lc)) > M_max:
        neighbors(e, lc) = SELECT_NEIGHBORS(e, neighbors(e, lc), M_max, lc)
    ep = closest_in(cands)

if l > L:
    entry_point = q

```

The Neighbor Selection Heuristic (Diversity-Aware)

A naive approach would simply pick the M closest vectors from candidates. HNSW uses a smarter heuristic that favors diversity: it checks whether a candidate c is closer to the new node q than it is to any already-selected neighbor. This prevents selecting a cluster of mutually-close vectors, ensuring selected neighbors span different directions and improving graph connectivity.

```

function SELECT_NEIGHBORS(q, W, M, lc):
    R = []
    W_d = sorted(W, by=dist(q, w), ascending=True)
    for e in W_d:
        if R is empty:
            R.append(e)           // always add the closest
        else:
            if dist(q, e) < min(dist(e, r) for r in R):
                R.append(e)       // only if diverse enough
            if len(R) == M: break
    return R

```

4.5 HNSW Delete Algorithm

Deletion is the most complex HNSW operation. There is no clean way to remove a node and repair the graph perfectly without substantial work.

Approach 1: Lazy Deletion (Tombstoning)

Mark the node as deleted. It remains in the graph but is filtered from results. Simple and $O(1)$, but dead nodes accumulate: they consume memory, slow traversal, and degrade graph quality over time.

- Pros: $O(1)$ delete, no graph modification, simple to implement
- Cons: Memory not freed; search quality degrades; periodic full rebuilds required

Approach 2: Repair-Based Deletion

6. Remove all edges involving the deleted node at every layer.

7. For each former neighbor: find its new nearest neighbors (excluding the deleted node) and reconnect.
8. Update the global entry point if the deleted node was the entry point.

This is correct but expensive — $O(M^2 \log n)$ per deletion. Most production systems implement lazy deletion with background compaction: when the deleted fraction exceeds ~15-20%, a full index rebuild is scheduled.

4.6 HNSW Parameters and Tuning

Parameter	Default	Effect
M	16	Max edges per node per layer. Higher M → better recall, more memory, slower insert
M_max	32 (L0)	Max edges at layer 0. Often $2 \times M$ for denser base layer
ef_construction	200	Candidate set during build. Higher → better graph quality, slower build
ef_search	50	Candidate set at query time. Higher → better recall, slower queries
mL	$1/\ln(M)$	Layer distribution multiplier. Rarely changed from default

The most impactful runtime lever is ef_search. No amount of ef_search tuning can recover from a poorly built index — M and ef_construction set the quality ceiling.

4.7 Time and Space Complexity

Operation	Time Complexity	Notes
Search	$O(\log n)$	Amortized; constant factor depends on M and ef
Insert	$O(M \times \log n)$	Dominated by neighbor search at each layer
Delete (lazy)	$O(1)$	Just a flag update
Delete (repair)	$O(M^2 \times \log n)$	Reconnecting M neighbors, each requiring M-length search
Full build (n vectors)	$O(n \times M \times \log n)$	Sequential inserts

Graph memory overhead (excluding raw vectors):

Graph memory $\approx n \times M \times 2 \times \text{layer_factor} \times 4$ bytes
Example: 1M vectors, $M=16 \rightarrow \sim 145$ MB graph overhead

4.8 Why HNSW Dominates in Practice

- Incremental inserts: Add new vectors without rebuilding the entire index. IVF-based indexes require full rebuilds.
- No training phase: Unlike IVF, no k-means clustering is needed before building. Start inserting immediately.
- Runtime tuning: `ef_search` adjusts recall/speed per-query without rebuilding.
- Strong benchmarks: ann-benchmarks.com consistently shows HNSW achieving 95%+ recall at thousands of QPS on commodity hardware.

Chapter 5: Vector Database Architecture

A vector database is not just an ANN index. It is a complete data management system with storage, indexing, query processing, filtering, replication, and failure recovery. This chapter examines the full architecture of a production vector database, layer by layer.

5.1 The Three Core Layers

- **Storage Layer:** How raw vectors and metadata are physically stored and persisted
- **Index Layer:** The data structures (HNSW) that enable fast approximate search
- **Query Layer:** The processing pipeline that handles incoming queries, filtering, ranking, and results

5.2 The Storage Layer

Write-Ahead Log (WAL)

Every write goes first to a write-ahead log — a sequential append-only file. This guarantees durability: even if the server crashes mid-write, the WAL can be replayed on restart to restore a consistent state. This is the same pattern used by PostgreSQL, RocksDB, and virtually every production database.

In-Memory Segment (Memtable)

Recent writes live in a fast in-memory memtable. Queries against recent data hit the memtable first, making fresh data immediately searchable. The memtable is backed by the WAL and can be rebuilt from it after a crash. Once it reaches a size threshold it is flushed to disk as an immutable segment.

Immutable On-Disk Segments

Flushed segments are never modified. This design enables lock-free reads, safe compaction, and crash resilience. Background compaction jobs periodically merge old segments and purge tombstoned (deleted) vectors.

Vector Compression in Storage

- **No compression (flat):** Maximum accuracy, maximum storage cost
- **Scalar Quantization (SQ8):** 4x reduction, under 1% recall loss — recommended default
- **Product Quantization (PQ):** 32x+ reduction, meaningful recall loss, used under severe memory constraints
- **Matryoshka dimensions:** Modern models can truncate dimensions (e.g., 3072 → 512) with moderate accuracy loss

5.3 The Index Layer

Index Build Strategy

- Online/incremental: Each vector is inserted into HNSW immediately. Always searchable, but slower overall.
- Batch/offline: Vectors are collected in bulk and the index is built from scratch. Faster, but unavailable during construction.

Index Tiering

- Hot tier: Recent or frequently accessed vectors in-memory with full HNSW
- Warm tier: Older data on SSD with memory-mapped HNSW
- Cold tier: Archival data on object storage, searched only on explicit request

5.4 The Query Layer

Standard Query Pipeline

9. Receive query vector (or raw text to embed)
10. Apply pre-filter if a metadata filter is specified
11. Run ANN search against the HNSW index
12. Apply post-filter if pre-filter was skipped
13. Fetch full metadata for result vectors
14. Apply optional reranking
15. Return top-k results with similarity scores and metadata

5.5 Metadata Filtering: The Hard Problem

Most real queries include both vector similarity and metadata constraints: 'find the 10 most similar documents, but only from workspace_id=42 and created after 2024-01-01.' Combining ANN with arbitrary filters is one of the hardest engineering problems in vector database design.

Post-Filtering

Run ANN first (ignoring the filter), retrieve a large candidate set, then apply the filter.

- Pros: Simple; fast ANN phase; filter doesn't affect the index
- Cons: Highly selective filters (0.1% match rate) require fetching 1,000× too many candidates, defeating ANN

Pre-Filtering

Apply the metadata filter first to get eligible IDs, then run ANN restricted to those IDs.

- Pros: Guaranteed k results if k eligible vectors exist
- Cons: Very permissive filters require iterating millions of IDs; very selective filters fall back to brute-force anyway

In-Graph Filtering (Filtered HNSW)

Modify the HNSW traversal to skip nodes that don't match the filter. Used by Qdrant, Weaviate, and others.

- Pros: Correct results at all selectivity levels
- Cons: Very selective filters cause many dead-end traversals; requires careful fallback to brute-force

Production Rule of Thumb

Selectivity (fraction of vectors matching the filter) drives strategy:

- Selectivity > 20%: Post-filtering works fine
- Selectivity 1–20%: In-graph filtering preferred
- Selectivity < 1%: Pre-filter then brute-force on the filtered subset

Most mature systems implement adaptive strategies that choose dynamically.

5.6 Reranking

After ANN search, a reranker rescores each candidate with a more accurate (but more expensive) method:

- Exact distance reranking: Recompute distances with full-precision vectors. Cheap; recovers from quantization error.
- Cross-encoder reranking: A full language model scores query+result pairs. 100x more expensive but significantly more accurate.
- Hybrid/RRF reranking: Combine vector similarity scores with BM25 keyword scores using Reciprocal Rank Fusion.

Chapter 6: Production Operations

Getting a vector database working in a demo is easy. Getting it to sustain consistent latency, correct results, and graceful failure recovery under production load is a different discipline. This chapter covers what actually breaks and how to prevent it.

6.1 Scaling Strategies

Vertical Scaling (Scale-Up)

The simplest strategy is a larger machine. Vector databases benefit enormously from more RAM (keeping the full index in memory) and faster CPUs. Modern CPUs support SIMD (AVX-512) instructions that compute distances for 16 float32 values simultaneously — a hardware-level speedup most general-purpose databases don't exploit.

Horizontal Scaling (Sharding)

When a single machine can't hold the index in memory, shard across multiple nodes:

- Data sharding: Different vectors on different nodes. Each query is broadcast to all shards; each returns its local top-k; a coordinator merges and re-sorts. Standard approach in Pinecone, Milvus, etc.
- Replicated shards: Each shard replicated across multiple nodes for fault tolerance and read throughput.

The Scatter-Gather Problem

Sharding creates a scatter-gather overhead: a global top-10 query must scatter to all shards, each does ANN and returns local top-10, then the coordinator gathers all ($n_shards \times 10$) results and picks the global top-10. ANN accuracy also decreases on shards because the local HNSW graph only sees a fraction of the global dataset. Mitigation: over-fetch at each shard (request top-50 for a global top-10) and use larger ef values.

6.2 Replication and Consistency

- Strong consistency: All reads see the latest write. Requires coordination overhead. Not common in high-throughput vector systems.
- Eventual consistency: Reads may see slightly stale data. Acceptable for most vector use cases — users rarely notice 1-second staleness.
- Session consistency: A user always sees their own writes. Common middle ground.

6.3 Cost Structure

Cost Component	Driver	Optimization Strategy
Compute	$QPS \times \text{latency per query}$	Quantization, better hardware,

Cost Component	Driver	Optimization Strategy
		caching
Memory	Index size (graph + vectors)	PQ compression, tiered storage, reducing M
Storage	Total vector data on disk	SQ8 quantization, TTL/expiry policies
Network	Scatter-gather fan-out	Fewer shards, locality-aware routing

In most deployments, memory is the dominant cost driver. The HNSW graph for 10M vectors (M=16) requires ~2-3 GB RAM for graph structure, plus ~60 GB for raw float32 vectors. Applying SQ8 reduces this to ~15 GB; PQ can reduce it to ~2 GB.

6.4 Monitoring and Alerting

Query Performance

- p50, p95, p99 query latency — alert if p99 exceeds your SLA
- Queries per second (QPS) — track against capacity ceiling
- ANN recall rate — measure periodically against ground truth (expensive but critical)

Index Health

- Vectors stored vs. capacity ceiling
- Deleted vector fraction — schedule rebuild if above 20%
- Index build time — spikes indicate write volume anomalies

Resource Utilization

- Memory usage — alert above 80% to avoid OOM crashes
- CPU usage during query vs. during index building (competing workloads)
- WAL size — unbounded growth indicates a broken flush process

6.5 Common Failure Modes

Failure	Symptom	Fix
OOM crash	Node crashes without warning	Add quantization or sharding; keep index below 75% of RAM
Insert-time latency spike	p99 spikes during data ingestion	Separate write and read paths; rate-limit ingestion

Failure	Symptom	Fix
Recall degradation	Results become less relevant over time	Accumulated deletions degrading graph; rebuild index
Cold-start latency	First queries after restart very slow	Pre-warm index on startup; use mmap for persistence
Shard imbalance	One shard consistently slower	Re-shard or rebalance data distribution

6.6 Zero-Downtime Index Rebuilds

Index rebuilds are necessary when: deleted fraction exceeds threshold; M or ef_construction must change; embedding model is changed; quantization scheme is updated.

Standard Blue-Green Rebuild

16. Build the new index on a shadow node while the current index serves traffic.
17. Verify recall quality of the new index against the old index (run evaluation set against both).
18. Atomically cut over: route new queries to the new index; drain in-flight queries on the old.
19. Shut down the old index.

Rebuild time estimate for 10M vectors (M=16, ef_construction=200) on a modern 32-core server: approximately 1-4 hours depending on vector dimension and hardware.

Chapter 7: Choosing the Right System

You should not automatically reach for a dedicated vector database. The right choice depends on your dataset size, query patterns, latency requirements, and team expertise.

7.1 Decision Framework

Start with pgvector

pgvector is a PostgreSQL extension that adds HNSW and IVFFlat indexing to Postgres. If you already run Postgres, it lets you add semantic search without introducing a new database technology.

Use pgvector when:

- Dataset is under 500,000 vectors
- Query volume is under 100 QPS
- Latency requirement is under 100ms (not sub-10ms)
- You need to JOIN vector results with relational data
- Team is already comfortable with PostgreSQL

Move to a Dedicated Vector DB when:

- Dataset exceeds 1 million vectors with sub-50ms latency requirements
- Query volume exceeds 500 QPS on a single node
- Advanced filtering with high selectivity is needed (pgvector's filtered search degrades badly)
- Multi-tenant isolation at the index level is required
- Built-in replication and horizontal sharding are needed

7.2 System Comparison

Database	Type	Strengths	Weaknesses
pgvector	Postgres extension	No new infra; SQL joins; transactions	Performance ceiling; limited horizontal scale
Pinecone	Managed cloud	Zero ops; auto-scaling; serverless option	Expensive at scale; vendor lock-in; no self-host
Qdrant	Self-host / cloud	Excellent filtered search; Rust performance; OSS	Smaller ecosystem; newer than Weaviate/Milvus
Weaviate	Self-host / cloud	GraphQL API; built-in	Complex

Database	Type	Strengths	Weaknesses
		rerankers and modules	configuration; higher memory overhead
Milvus	Self-host	Billion-scale; DiskANN support; mature	Complex ops; heavyweight Kubernetes deployment
Chroma	Embedded / OSS	Easiest to start; great for prototyping	Not production-grade at scale
LanceDB	Embedded / cloud	Serverless; analytics + vectors; columnar format	Newer; smaller community

7.3 Hybrid Search

Hybrid search combines vector (semantic) similarity with keyword (BM25/TF-IDF) relevance. In production, it almost always outperforms pure vector search because:

- Vector search misses exact keyword matches (product codes, error messages, proper nouns)
- Keyword search misses semantic variations but is precise for exact terms
- Combining both covers the weakness of each approach

The standard combination method is Reciprocal Rank Fusion (RRF):

$$\text{RRF_score}(d) = \sum_i \frac{1}{(k + \text{rank}_i(d))}$$

k is typically 60; $\text{rank}_i(d)$ is the rank of document d in retrieval list i

RRF is rank-based (not score-based), making it robust to different score scales from different retrieval methods. Most production RAG systems use RRF as their default fusion strategy.

7.4 Embedding Model Selection

The embedding model choice has a larger impact on retrieval quality than the choice of vector database. A mediocre model with a great database will underperform a great model with a simple database.

20. Define your task: document retrieval, Q&A, code search, image search, multilingual?
21. Build an evaluation dataset of (query, relevant document) pairs from real production data
22. Benchmark candidate models using NDCG@10 or Recall@10 on this dataset
23. Factor in cost: API pricing, inference latency, dimension count

Resource: The MTEB (Massive Text Embedding Benchmark) leaderboard at huggingface.co/spaces/mteb/leaderboard provides comprehensive model comparisons across many tasks and languages.

7.5 Billion-Scale Systems

For datasets exceeding 100M vectors, HNSW's memory requirements become prohibitive (terabytes of RAM). Emerging approaches:

DiskANN

Developed by Microsoft Research, DiskANN stores the majority of the index on SSD rather than RAM. It achieves competitive recall and throughput for billion-scale datasets at a fraction of HNSW's memory cost. Now available in Azure AI Search, Milvus, and others.

Hierarchical IVF+HNSW

Cluster the dataset into groups; build a coarse HNSW over cluster centroids (fits in RAM); store per-cluster flat indexes on disk. A query first identifies relevant clusters, then searches within them. Conceptually similar to IVF but adapted for disk-based storage.

Chapter 8: Building a Complete Vector Search System

In this final chapter we walk through building a production-ready vector search system end to end: data ingestion, embedding, indexing, querying, hybrid search, and monitoring.

8.1 System Design

Our reference system is a document search engine for a knowledge base:

- 100,000 documents averaging 2,000 words (chunked to ~500 tokens → ~400,000 vectors)
- Query volume: 200 QPS peak
- Latency requirement: p99 < 50ms

Sizing conclusion: pgvector on a 16-core / 64 GB RAM machine handles this comfortably. This is our architecture.

8.2 Data Ingestion Pipeline

```
# Step 1: Chunk documents
from langchain.text_splitter import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=50,
    separators=['\n\n', '\n', '. ', ' ']
)
chunks = splitter.split_documents(documents)
```

```
# Step 2: Embed in batches (rate-limit aware)
import openai

def embed_batch(texts, model='text-embedding-3-small'):
    response = openai.embeddings.create(input=texts, model=model)
    return [r.embedding for r in response.data]

BATCH_SIZE = 100
all_embeddings = []
for i in range(0, len(chunks), BATCH_SIZE):
    batch = [c.page_content for c in chunks[i:i+BATCH_SIZE]]
    all_embeddings.extend(embed_batch(batch))
```

8.3 Schema and Index Setup

```
-- PostgreSQL + pgvector setup
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE document_chunks (
    id          BIGSERIAL PRIMARY KEY,
    doc_id      TEXT NOT NULL,
    chunk_index INTEGER NOT NULL,
    content     TEXT NOT NULL,
    embedding   vector(1536) NOT NULL,
    created_at  TIMESTAMPTZ DEFAULT NOW(),
    metadata    JSONB
);

-- HNSW index (cosine similarity)
CREATE INDEX ON document_chunks
    USING hnsw (embedding vector_cosine_ops)
    WITH (m = 16, ef_construction = 200);

-- Metadata filter indexes
CREATE INDEX ON document_chunks (doc_id);
CREATE INDEX ON document_chunks USING GIN (metadata);
CREATE INDEX ON document_chunks USING GIN (to_tsvector('english', content));
```

8.4 Vector Search Query

```
def semantic_search(query_text, k=10, filter=None):
    query_embedding = embed_batch([query_text])[0]
    vec_str = '[' + ','.join(str(x) for x in query_embedding) + ']'

    sql = '''
        SELECT id, doc_id, content, metadata,
               1 - (embedding <=> %s::vector) AS similarity
        FROM document_chunks
        WHERE ($2::jsonb IS NULL OR metadata @> $2)
        ORDER BY embedding <=> %s::vector
        LIMIT %s
    '''
    return db.execute(sql, [vec_str, json.dumps(filter), vec_str, k])
```

8.5 Hybrid Search with RRF

```
def hybrid_search(query_text, k=10):
    vector_results = semantic_search(query_text, k=k*3)

    keyword_sql = '''
```

```

        SELECT id FROM document_chunks
        WHERE to_tsvector('english', content) @@ plainto_tsquery('english',
%s)

        ORDER BY ts_rank(to_tsvector('english', content),
                        plainto_tsquery('english', %s)) DESC

        LIMIT %s
    """
    keyword_results = db.execute(keyword_sql, [query_text, query_text, k*3])

    K = 60                                # RRF constant
    scores = {}
    for rank, row in enumerate(vector_results):
        scores[row.id] = scores.get(row.id, 0) + 1 / (K + rank + 1)
    for rank, row in enumerate(keyword_results):
        scores[row.id] = scores.get(row.id, 0) + 1 / (K + rank + 1)

    top_ids = sorted(scores, key=scores.get, reverse=True)[:k]
    return fetch_chunks_by_ids(top_ids)

```

8.6 Recall Evaluation

```

def evaluate_recall(test_queries, ground_truth, k=10):
    """
    test_queries: list of query strings
    ground_truth: dict mapping query -> set of relevant doc IDs
    Returns: mean Recall@k
    """
    recalls = []
    for query in test_queries:
        results = semantic_search(query, k=k)
        result_ids = {r.id for r in results}
        relevant = ground_truth[query]
        recall = len(result_ids & relevant) / len(relevant)
        recalls.append(recall)
    return sum(recalls) / len(recalls)

```

Run this evaluation on 200-500 representative queries whenever you change the embedding model, adjust HNSW parameters, or significantly modify chunking strategy. A recall drop of more than 3-5% signals a regression.

8.7 Monitoring

```

from prometheus_client import Histogram, Counter, Gauge
import time

query_latency = Histogram('vdb_query_seconds', 'Query latency',
    buckets=[.005, .01, .025, .05, .1, .25, .5, 1, 2.5])
query_count = Counter('vdb_queries_total', 'Total queries')

```

```
index_size    = Gauge('vdb_vectors_total', 'Vectors in index')

def instrumented_search(query_text, k=10):
    query_count.inc()
    start = time.time()
    try:
        return semantic_search(query_text, k)
    finally:
        query_latency.observe(time.time() - start)
```

8.8 What Comes Next

- Multi-vector search: Store multiple embeddings per chunk (dense + sparse) for higher accuracy
- Late interaction models (ColBERT): Token-level embeddings with MaxSim scoring — significantly higher recall for complex queries
- Embedding fine-tuning: Fine-tune on domain-specific data for specialized content (often 10-30% recall improvement)
- Query caching: Cache embedding computations and results for repeated queries — large cost savings at scale
- Tenant isolation: Separate namespaces or collections per customer to prevent cross-tenant data leakage

Vector databases are a rapidly evolving field. The algorithms and systems in this book represent the state of the art as of 2025. New approaches for billion-scale datasets and multimodal embeddings are emerging every year. The fundamentals — distance geometry, graph traversal, quantization tradeoffs, and distributed systems design — will remain relevant regardless of which specific system wins.

— END OF BOOK —